

MC833 - Relatório Trabalho 01

Raw

Yvens Ian Prado Porto - RA 184031

1 Introdução

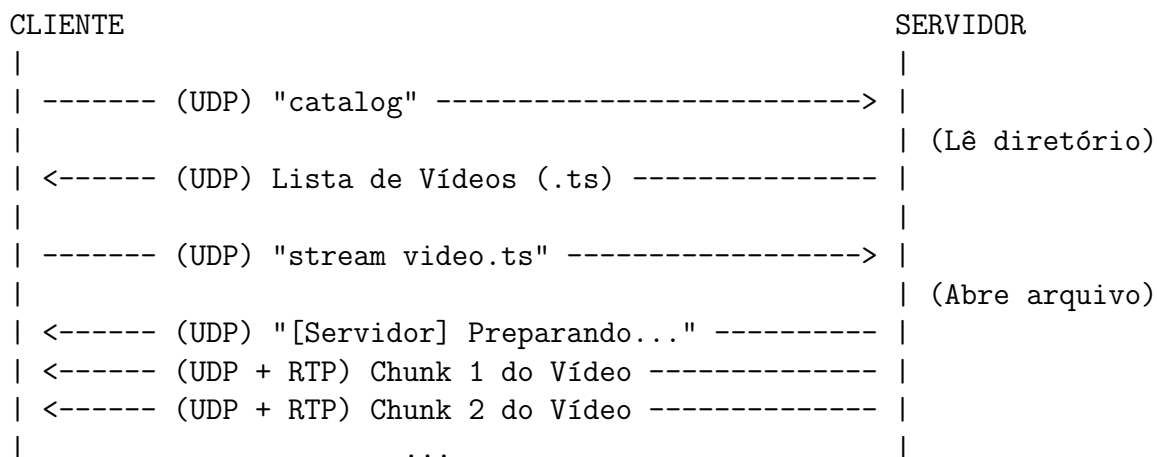
Este relatório descreve o desenvolvimento da primeira fase do Trabalho 01 da disciplina MC833. O objetivo principal foi implementar uma aplicação de streaming de vídeo no modelo Cliente-Servidor utilizando *Raw Sockets* na linguagem Python. Ao contornar a pilha de protocolos padrão do sistema operacional, foi necessário realizar a construção manual e o encapsulamento de todos os cabeçalhos das camadas de Rede e Transporte (IP e UDP), além da implementação do protocolo de aplicação RTP para a transmissão contínua do vídeo.

2 Protocolo e Diagrama de Sequência

A aplicação utiliza o protocolo UDP na camada de transporte, em conjunto com o protocolo RTP na camada de aplicação.

A escolha do UDP justifica-se pela natureza do streaming de vídeo em tempo real, onde a entrega rápida dos dados é mais importante do que a confiabilidade absoluta. O TCP introduziria atrasos inaceitáveis (*head-of-line blocking*) devido às retransmissões automáticas de pacotes perdidos. No entanto, como o UDP não garante ordem nem sincronização, o RTP foi escolhido para encapsular o vídeo. O cabeçalho RTP preenche essas lacunas ao prover um *Sequence Number* (permitindo ao cliente reordenar pacotes ou detectar perdas) e um *Timestamp* (essencial para a reprodução do vídeo na velocidade correta e cálculo de *jitter*).

Abaixo, o Diagrama de Sequência ilustrando a comunicação:



```

| <----- (UDP + RTP) Chunk N do Vídeo ----- |
| <----- (UDP) "[Servidor] EOF" ----- |
|

```

3 Estrutura dos Cabeçalhos e Alocação de Bytes

Para cada pacote de vídeo transmitido, a aplicação monta a seguinte estrutura de cabeçalhos (*headers*) antes do envio na rede. O tamanho total de cabeçalhos por pacote é de 40 bytes.

Protocolo	Tamanho	Campos Reservados
IPv4	20 bytes	Version, IHL, TOS, Total Length, ID, Flags, Fragment Offset, TTL, Protocol (17), Header Checksum, Source IP, Dest IP.
UDP	8 bytes	Source Port, Destination Port, Length, Checksum.
RTP	12 bytes	Version, Padding, Extension, CSRC Count, Marker, Payload Type (33), Sequence Number, Timestamp, SSRC.

Tabela 1: Estrutura dos cabeçalhos construídos via struct do Python.

3.1 Bytes Reservados para Dados (Payload)

A quantidade de bytes reservada exclusivamente para os dados (o vídeo em si) em cada pacote é de 1316 bytes. Esse valor foi escolhido com base na arquitetura do *MPEG Transport Stream* (.ts), onde cada pacote TS possui exatos 188 bytes. O valor de 1316 bytes permite encapsular de forma perfeitamente alinhada 7 pacotes TS por *datagrama* UDP ($188 \times 7 = 1316$), evitando fragmentação desnecessária no nível de aplicação.

4 Análise de Desempenho e Taxa de Transmissão

Para calcular a quantidade de pacotes por *frame* e a taxa de transmissão exigida pela rede, adotamos a premissa de que os vídeos operam a uma taxa de 30 frames por segundo (fps).

4.1 Metodologia de Cálculo

1. Total de Frames: Duração (s) $\times$ 30 fps.
2. Bytes por Frame (S_f): Tamanho do Arquivo (em bytes)/Total de Frames.
3. Pacotes por Frame (P): $\lceil S_f/1316 \text{ bytes} \rceil$ (arredondado para o próximo inteiro).
4. Taxa de Transmissão (T): Sabendo que o tamanho total de um pacote na rede (Dados + IP + UDP + RTP) é de 1356 bytes, a taxa em bits por segundo (bps) é: $T = P \times 1356 \times 8 \times 30$.

Para fins de precisão na conversão de armazenamento, utilizou-se $1 \text{ MB} = 1.048.576 \text{ bytes}$. A seguir, os resultados para os três vídeos testados no catálogo.

4.2 Vídeo 1: edu.ts

Especificações: Tamanho: 7.2 MB (7.549.747 bytes) — Duração: 14 s — Total de Frames: 420.

- Tamanho médio por frame: ≈ 17.975 bytes.
- Quantidade de pacotes por frame: 14 pacotes.
- Taxa de transmissão requerida (30fps): ≈ 4.55 Mbps (4.556.160 bps).

4.3 Vídeo 2: golden.ts

Especificações: Tamanho: 165.1 MB (173.119.897 bytes) — Duração: 198 s — Total de Frames: 5940.

- Tamanho médio por frame: ≈ 29.144 bytes.
- Quantidade de pacotes por frame: 23 pacotes.
- Taxa de transmissão requerida (30fps): ≈ 7.48 Mbps (7.485.120 bps).

4.4 Vídeo 3: sioiosikis.ts

Especificações: Tamanho: 78.3 MB (82.103.500 bytes) — Duração: 58 s — Total de Frames: 1740.

- Tamanho médio por frame: ≈ 47.185 bytes.
- Quantidade de pacotes por frame: 36 pacotes.
- Taxa de transmissão requerida (30fps): ≈ 11.71 Mbps (11.715.840 bps).

5 Resultados Finais

O ambiente foi validado com êxito. A escolha pelo *Transport Stream* foi motivada pela sua arquitetura, que diferentemente de arquivos **.mp4**, permite a leitura e reprodução contínua e sequencial sem a dependência de um índice central (*moov atom*).

O sistema provou sua integridade tanto de forma passiva (verificando a semelhança absoluta de *hashs* MD5 entre o arquivo origem no contêiner do Servidor e o recebido no contêiner do Cliente) quanto ativamente: foi possível iniciar um *player* de vídeo apontado para o arquivo em construção na máquina *host* e assistir à transmissão em tempo real simultânea ao seu processamento na rede. O sistema de tratamento de exceções do *Servidor* atuou corretamente em requisições de vídeos inexistentes, evitando o travamento do *Cliente*.

6 Declaração de Uso de Inteligência Artificial

Em conformidade com as diretrizes da disciplina, declaro que ferramentas de Inteligência Artificial Generativa foram utilizadas durante o desenvolvimento deste trabalho como assistentes de tutoria e *pair-programming*.

Justificativa e Detalhamento de Uso:

- **Empacotamento de Bits:** Auxílio na compreensão das máscaras de bits (*bitwise operations*) e nos formatos da biblioteca `struct` do Python para a montagem exata dos cabeçalhos RTP, UDP e IPv4.
- **Depuração de Redes (*Debugging*):** Suporte na identificação do comportamento de *Hanging Client* (cliente travado aguardando resposta) e na lógica de extração dos 14 bytes iniciais do *frame Ethernet* da camada de enlace ao utilizar o *socket AF_PACKET*.
- **Estruturação Matemática e $\text{\LaTeX}$:** Auxílio na formatação do relatório técnico e na montagem das equações para o cálculo formal da taxa de transmissão de rede (*throughput*) e pacotes por *frame*.

Ressalto que toda a arquitetura base, execução no ambiente Docker, testes de conceito, validação de integridade via *hashes* MD5 e a reprodução real do fluxo de vídeo foram conduzidos, executados e interpretados de forma autoral na minha máquina local.